

Programação de Algoritmos de Compressão e Análise de Imagens

Relatório do Trabalho Prático 2

Enzo Borges Segala, Jonathan Gabriel Nunes Mendes, Matheus Machado Cezar,
Marcos Henrique Volpato de Moraes, Miguel Predebon Abichequer, Nicolas Rosenthal
Dal Corso, and Rafael Silveira Bandeira

Universidade Federal do Rio Grande do Sul

11/11/2025

Sumário

1	Compressão baseada no modo básico do JPEG	3
1.1	Metodologia	3
1.1.1	Preparação da imagem	3
1.1.2	Discrete Cosine Transform (DCT)	3
1.1.3	Quantização	3
1.1.4	Reconstrução (Descompressão)	3
1.1.5	Cálculo das métricas	3
1.2	Código	4
1.3	Resultados Obtidos	7
1.3.1	Imagens originais	7
1.3.2	Imagens resultantes	8
1.4	Análise dos Resultados	9
2	Compressão baseada na quantização vetorial	11
2.1	Metodologia	11
2.1.1	Formação dos blocos	11
2.1.2	Construção do Codebook (K-Means)	11
2.1.3	Compressão	11
2.1.4	Reconstrução	11
2.1.5	Métricas	11
2.2	Código	11
2.3	Resultados Obtidos	15
2.4	Análise dos Resultados	15
3	Segmentação	16
3.1	Metodologia	16
3.1.1	Segmentação por K-means no Espaço de Cor Lab	16
3.1.2	Segmentação baseada em Textura com Filtros de Gabor	16
3.2	Código	17
3.3	Resultados Obtidos e Análise	17
3.3.1	Imagens originais	17
3.3.2	Borboleta K-Means	17
3.3.3	Borboleta Gabor + K-Means	17
3.3.4	Raposa K-Means	18
3.3.5	Raposa Gabor + K-Means	18
3.4	Conclusão da Análise	18
	Siglas	19

1 Compressão baseada no modo básico do JPEG

Nesta etapa do trabalho foi implementada uma versão simplificada do método JPEG baseline, mantendo as operações essenciais do algoritmo, porém sem utilizar a etapa de codificação entrópica (Run-Length Encoding e codificação de Huffman). O objetivo é analisar o comportamento da compressão apenas com as etapas de transformação, quantização e reconstrução.

1.1 Metodologia

A compressão foi aplicada a duas imagens em tons de cinza selecionadas previamente. O método adotado segue a lógica do JPEG básico, composta pelas seguintes etapas:

1.1.1 Preparação da imagem

A imagem original é convertida para valores do tipo double e, caso suas dimensões não sejam múltiplas de 8, aplica-se padding replicando os valores da borda. Isso garante que a imagem possa ser dividida integralmente em blocos de 8×8 pixels, tamanho padrão do JPEG.

1.1.2 Discrete Cosine Transform (DCT)

Após a divisão em blocos, é aplicada a DCT bidimensional em cada bloco. Essa transformada converte a informação espacial da imagem em componentes de frequência, concentrando grande parte da energia nos primeiros coeficientes.

1.1.3 Quantização

Cada bloco transformado é quantizado utilizando uma matriz de quantização padrão do JPEG, escalada por um fator Q , que controla a intensidade da compressão. Valores maiores de Q provocam quantização mais agressiva e, portanto, maior perda de informação e maior taxa de compressão.

1.1.4 Reconstrução (Descompressão)

Para reconstruir a imagem comprimida, os coeficientes quantizados são de-quantizados (multiplicação pela matriz), e a Inverse Discrete Cosine Transform (IDCT) é aplicada aos blocos, retornando ao domínio espacial.

1.1.5 Cálculo das métricas

Dois indicadores foram utilizados para avaliar os resultados:

- **Taxa de compressão:** estimada considerando a proporção entre o número total de coeficientes e o número de coeficientes não nulos após a quantização.
- **Peak Signal-to-Noise Ratio (PSNR):** mede a similaridade entre a imagem original e a reconstruída; valores maiores indicam menor perda de qualidade.

1.2 Código

```
function [rec_img, ratio, psnr_val] = jpeg_basic(img, Q)

img = double(img);
[h, w] = size(img);

% Ajustar dimensões para múltiplos de 8
H = ceil(h/8) * 8;
W = ceil(w/8) * 8;

padded = padarray(img, [H-h W-w], 'replicate', 'post');

% Matriz de quantização base do JPEG
Qbase = [16 11 10 16 24 40 51 61;
         12 12 14 19 26 58 60 55;
         14 13 16 24 40 57 69 56;
         14 17 22 29 51 87 80 62;
         18 22 37 56 68 109 103 77;
         24 35 55 64 81 104 113 92;
         49 64 78 87 103 121 120 101;
         72 92 95 98 112 100 103 99];

Qmatrix = Q * Qbase;

coeffs = zeros(H,W);

% --- Compressão: DCT + Quantização ---
for i = 1:8:H
    for j = 1:8:W
        bloco = padded(i:i+7, j:j+7);
        D = dct2(bloco);
        C = round(D ./ Qmatrix);
        coeffs(i:i+7, j:j+7) = C;
    end
end

% Estimar taxa de compressão
total = numel(coeffs);
nzeros = nnz(coeffs);
ratio = total / nzeros;

% --- Descompressão ---
rec = zeros(H,W);
for i = 1:8:H
    for j = 1:8:W
```

```

        C = coeffs(i:i+7, j:j+7);
        D = C .* Qmatrix;
        rec(i:i+7, j:j+7) = idct2(D);
    end
end

rec_img = rec(1:h, 1:w);

% --- PSNR ---
mse = mean((img(:) - rec_img(:)).^2);
psnr_val = 10 * log10(255^2 / mse);

end

warning('off', 'MATLAB:MKDIR:DirectoryExists'); % i know it may exists
mkdir('.out');

% Carregar imagens em tons de cinza
img1 = imread('borboleta.jpg');
img2 = imread('raposa.jpg');

if size(img1, 3) == 3
    img1 = rgb2gray(img1);
end

if size(img2, 3) == 3
    img2 = rgb2gray(img2);
end

% Valores de Q para teste
Qvalues = [2 4 8 16];

% =====
%   MOSTRAR APENAS UMA FIGURA COM AS DUAS IMAGENS ORIGINAIS
% =====
figure('Name', 'Imagens Originais', 'NumberTitle', 'off');
subplot(1, 2, 1);
imshow(uint8(img1));
title('Imagem 1 - Original (P&B)');

subplot(1, 2, 2);
imshow(uint8(img2));
title('Imagem 2 - Original (P&B)');

% =====
%   PROCESSAR E ARMAZENAR OS RESULTADOS PARA NÃO ABRIR MIL FIGURAS

```

```

% =====

rec_imgs1 = cell(length(Qvalues), 1);
rec_imgs2 = cell(length(Qvalues), 1);

ratios1 = zeros(length(Qvalues), 1);
ratios2 = zeros(length(Qvalues), 1);

psnr1_vals = zeros(length(Qvalues), 1);
psnr2_vals = zeros(length(Qvalues), 1);

for k = 1:length(Qvalues)

    Q = Qvalues(k);
    fprintf('\n== Testando com Q = %d ==\n', Q);

    % Imagem 1
    [rec1, ratio1, psnr1] = jpeg_basic(img1, Q);
    rec_imgs1{k} = rec1;
    ratios1(k) = ratio1;
    psnr1_vals(k) = psnr1;

    fprintf('Imagem 1 - Taxa: %.2f | PSNR: %.2f dB\n', ratio1, psnr1);

    % Imagem 2
    [rec2, ratio2, psnr2] = jpeg_basic(img2, Q);
    rec_imgs2{k} = rec2;
    ratios2(k) = ratio2;
    psnr2_vals(k) = psnr2;

    fprintf('Imagem 2 - Taxa: %.2f | PSNR: %.2f dB\n', ratio2, psnr2);

end

% =====
% MOSTRAR APENAS 1 FIGURA COM AS 4 COMPRESSÕES DA IMAGEM 1
% =====
figure('Name', 'Compressões - Imagem 1', 'NumberTitle', 'off');

for k = 1:length(Qvalues)
    subplot(2, 2, k);
    imshow(uint8(rec_imgs1{k}));
    title(['Q = ' num2str(Qvalues(k)) ...
          ' | PSNR = ' num2str(psnr1_vals(k), '%.2f') ...
          ' | Taxa = ' num2str(ratios1(k), '%.2f')]);
    imwrite(uint8(rec_imgs1{k}), strcat('.out/borboleta_', num2str(

```

```

        Qvalues(k)), '.png'));
end

% =====
%   MOSTRAR APENAS 1 FIGURA COM AS 4 COMPRESSÕES DA IMAGEM 2
%   =====
figure('Name', 'Compressões - Imagem 2', 'NumberTitle', 'off');

for k = 1:length(Qvalues)
    subplot(2, 2, k);
    imshow(uint8(rec_imgs2{k}));
    title(['Q = ' num2str(Qvalues(k)) ...
          ' | PSNR = ' num2str(psnr2_vals(k), '%.2f') ...
          ' | Taxa = ' num2str(ratios2(k), '%.2f')]);
    imwrite(uint8(rec_imgs2{k}), strcat('.out/raposa_', num2str(
        Qvalues(k)), '.png'));
end

```

1.3 Resultados Obtidos

A compressão foi aplicada utilizando $Q \in \{2, 4, 8 \text{ e } 16\}$ como o fator de quantização, de modo a observar como o aumento da quantização influencia tanto a taxa de compressão quanto a qualidade final da imagem reconstruída.

1.3.1 Imagens originais



Figura 1: Imagens originais

1.3.2 Imagens resultantes

Imagem 1: borboleta.jpg		
Q	Taxa de Compressão	PSNR (dB)
2	11.89	31.95
4	20.82	29.64
8	35.33	27.24
16	52.01	24.42



(a) $Q = 2$



(b) $Q = 4$



(c) $Q = 8$



(d) $Q = 16$

Figura 2: Resultado da aplicação na imagem borboleta

Imagem 2: raposa.jpg		
Q	Taxa de Compressão	PSNR (dB)
2	12.55	29.38
4	20.58	27.80
8	34.36	26.03
16	52.65	23.65

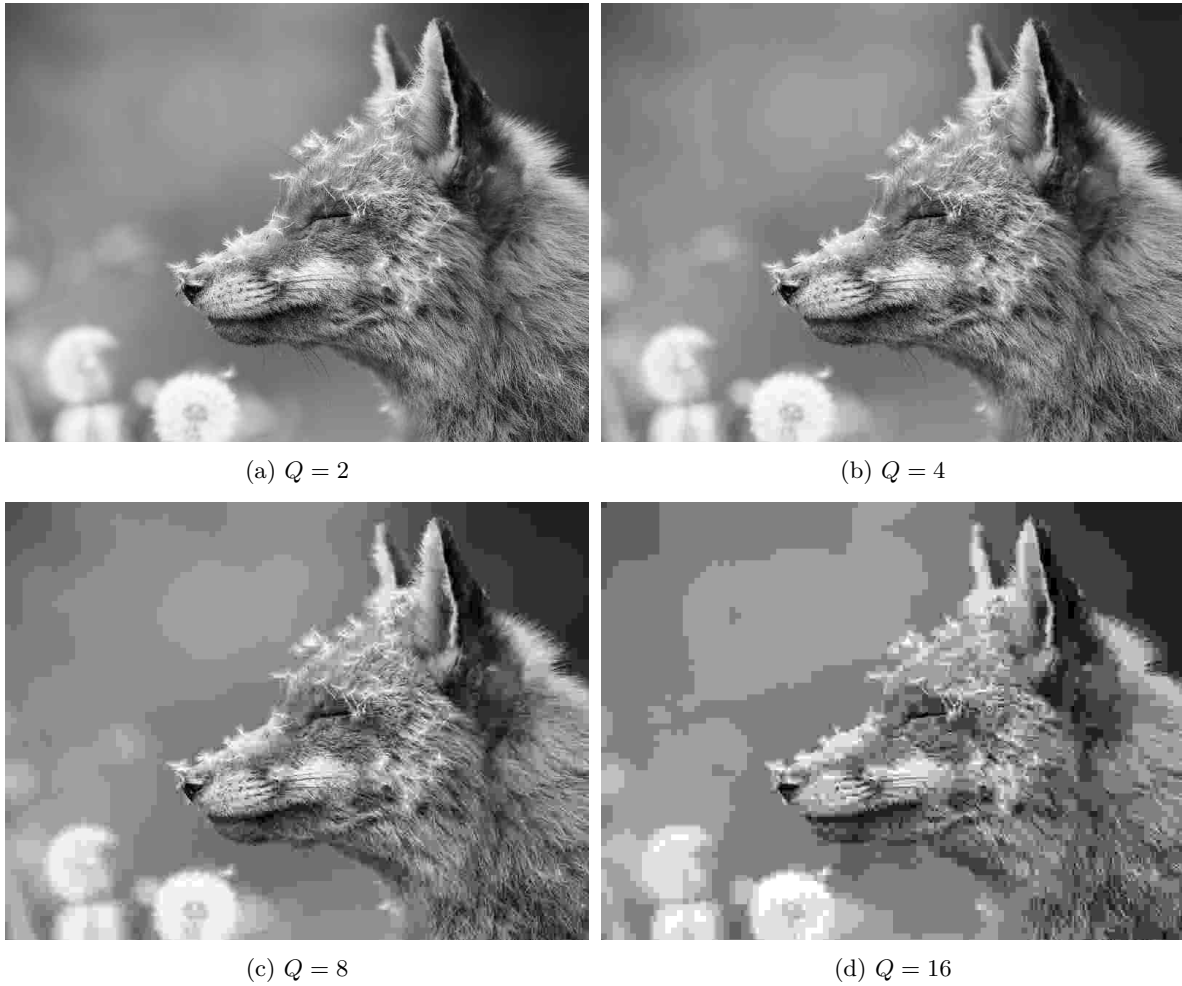


Figura 3: Resultado da aplicação na imagem raposa

1.4 Análise dos Resultados

Os testes realizados mostram claramente a relação entre o fator de quantização e a qualidade final da imagem:

- **Q baixo (ex.: $Q = 2$):** Quando a quantização é fraca, a perda de informação é pequena. As duas imagens apresentaram PSNR acima de 29 dB, indicando boa preservação visual. A taxa de compressão, porém, permanece relativamente modesta (cerca de 12 %).
- **Q intermediário ($Q = 4$ e 8):** À medida que o Q aumenta, mais coeficientes da DCT são aproximados para zero, o que reduz o tamanho dos dados. As taxas aumentam para cerca de 20 % e depois para mais de 30 %, ao custo de uma queda gradual no PSNR. Nessa faixa, os

primeiros artefatos típicos do JPEG começam a aparecer, especialmente blocagem em áreas com pouca textura.

- **Q alto ($Q = 16$):** A compressão torna-se bastante agressiva: as taxas ultrapassam 50 %, ou seja, mais da metade dos coeficientes se tornam nulos. Consequentemente, a qualidade se degrada mais intensamente, com PSNR entre 23 dB e 24 dB, valores que normalmente já representam perda de detalhes e blocos visíveis.

Comparando as duas imagens, nota-se que:

- A **Imagem 1** mantém valores de PSNR ligeiramente superiores em todos os níveis de compressão, sugerindo que sua estrutura visual é mais adequada ao modelo JPEG.
- A **Imagem 2** apresenta queda de qualidade um pouco mais acentuada, indicando sensibilidade maior à quantização, possivelmente por conter áreas suaves ou gradientes que são mais afetados por perdas nas altas frequências.

No geral, os resultados confirmam o comportamento esperado do algoritmo JPEG: **mais compressão** $\rightarrow$ **mais perda** $\rightarrow$ **menor PSNR**, com artefatos cada vez mais perceptíveis.

2 Compressão baseada na quantização vetorial

Nesta etapa do trabalho, foi implementada uma técnica de compressão baseada em **quantização vetorial (VQ)**. Esse método consiste em dividir a imagem em blocos de dimensões fixas, agrupar blocos similares por meio do algoritmo **K-Means**, e substituir cada bloco da imagem por um índice que referencia um bloco representativo dentro de um codebook.

O processo foi aplicado a quatro imagens: duas em tons de cinza (**doca** e **cachorro**) e duas coloridas (**borboleta** e **raposa**). Para cada imagem, foram testados dois tamanhos de blocos: 2×2 e 3×3 . Em todos os casos, o codebook é composto por **256 vetores**, conforme solicitado no enunciado.

2.1 Metodologia

O método foi dividido nas seguintes etapas:

2.1.1 Formação dos blocos

A imagem é percorrida em blocos **não sobrepostos**, de tamanho 2×2 ou 3×3 . Cada bloco é convertido em um vetor coluna. No caso de imagens coloridas, esse processo é feito **por canal (R,G,B)**.

2.1.2 Construção do Codebook (K-Means)

Coletamos uma amostra dos blocos da imagem e aplicamos o algoritmo **K-Means** para agrupá-los em 256 grupos. O centro de cada cluster torna-se uma **palavra do codebook**.

2.1.3 Compressão

Cada bloco da imagem é substituído por **um índice inteiro entre 1 e 256**, correspondente ao bloco mais próximo do codebook. A imagem comprimida contém: o codebook e a matriz de índices.

2.1.4 Reconstrução

A imagem é reconstituída substituindo cada índice pelo bloco correspondente no codebook. Para imagens coloridas, a operação é feita separadamente para os três canais.

2.1.5 Métricas

Foram calculados:

- **Taxa de compressão** = bits da imagem original / bits da representação VQ
- **PSNR**, comparando a imagem reconstruída com a original

2.2 Código

```
function [rec_img, ratio, psnr_med] = vq_color(img, B)

    R = img(:,:,1);
    G = img(:,:,2);
```

```

Bc = img(:,:,3);

[recR, rR, pR] = vq_gray(R, B);
[recG, rG, pG] = vq_gray(G, B);
[recB, rB, pB] = vq_gray(Bc, B);

rec_img = cat(3, recR, recG, recB);

ratio = (rR + rG + rB) / 3;
psnr_med = (pR + pG + pB) / 3;

end

function [rec_img, ratio, psnr_val] = vq_gray(img, B)

    img = double(img);

    % GARANTIA DE QUE É CINZA (2D)
    if ndims(img) ~= 2
        img = rgb2gray(uint8(img));
        img = double(img);
    end

    [h, w] = size(img);

    H = ceil(h/B) * B;
    W = ceil(w/B) * B;

    padded = padarray(img, [H-h, W-w], 'replicate', 'post');

    % EXTRAI BLOCOS SEMPRE 2D
    blocks = im2col(padded, [B B], 'distinct');
    N = size(blocks, 1);

    max_samples = 10000;

    if N > max_samples
        idx = randperm(N, max_samples);
        sample_blocks = blocks(idx, :);
    else
        sample_blocks = blocks;
    end

    K = 256;

    [~, codebook] = kmeans(sample_blocks, K, 'MaxIter', 1000);

```

```

D = pdist2(blocks, codebook);
[~, compressed_idx] = min(D, [], 2);

rec_blocks = codebook(compressed_idx, :);
rec_blocks = rec_blocks';

rec_padded = col2im(rec_blocks, [B B], [H W], 'distinct');

rec_img = rec_padded(1:h, 1:w);

bits_original = numel(img) * 8;
bits_codebook = numel(codebook) * 8;
bits_indices = N * 8;

bits_total = bits_codebook + bits_indices;

ratio = bits_original / bits_total;

mse = mean((img(:) - rec_img(:)).^2);
psnr_val = 10 * log10(255^2 / mse);

end

clear all; close all; clc;

gray_imgs = {'./out/doca.png', './out/cachorro.png'};
color_imgs = {'borboleta.jpg', 'raposa.jpg'};

B_values = [2 3];

% IMAGENS EM CINZA
for b = 1:length(B_values)
    B = B_values(b);

    figure('Name', ['Cinza B=' num2str(B)], 'NumberTitle', 'off');

    for i = 1:length(gray_imgs)

        img = imread(gray_imgs{i});

        if ndims(img) == 3
            img = rgb2gray(img);
        end

        img = double(img);
    end
end

```

```

[rec, ratio, psnr_val] = vq_gray(img, B);

disp(['Cinza | ' gray_imgs{i} ...
      ' | B=' num2str(B) ...
      ' | TAXA=' num2str(ratio, '%.2f') ...
      ' | PSNR=' num2str(psnr_val, '%.2f')]);

subplot(2, 2, i);
imshow(uint8(img));

subplot(2, 2, i + 2);
imshow(uint8(rec));

record(gray_imgs{i}, rec, B, ratio, psnr_val)
end

end

% IMAGENS COLORIDAS
for b = 1:length(B_values)
    B = B_values(b);

    figure('Name', ['Coloridas B=' num2str(B)], 'NumberTitle', 'off');

    for i = 1:length(color_imgs)

        img = imread(color_imgs{i});
        img = double(img);

        [rec, ratio, psnr_val] = vq_color(img, B);

        disp(['Color | ' color_imgs{i} ...
              ' | B=' num2str(B) ...
              ' | TAXA=' num2str(ratio, '%.2f') ...
              ' | PSNR=' num2str(psnr_val, '%.2f')]);

        subplot(2, 2, i);
        imshow(uint8(img));

        subplot(2, 2, i + 2);
        imshow(uint8(rec));

        record(color_imgs{i}, rec, B, ratio, psnr_val)
    end
end

```

end

2.3 Resultados Obtidos

Os valores de **taxa de compressão** e **PSNR** obtidos estão apresentados nas tabelas a seguir.

Imagens em tons de cinza			
Bloco	Imagem	Taxa de Compressão	PSNR (dB)
2×2	doca	3.92	37.30
	cachorro	3.92	40.24
3×3	doca	8.16	33.46
	cachorro	8.13	37.64

Imagens coloridas			
Bloco	Imagem	Taxa de Compressão	PSNR (dB)
2×2	borboleta	3.63	33.75
	raposa	3.63	34.82
3×3	borboleta	5.89	30.43
	raposa	5.89	31.23

2.4 Análise dos Resultados

Os resultados mostram claramente que o tamanho do bloco utilizado na VQ influencia diretamente a qualidade da reconstrução e a taxa de compressão. Com blocos de 2×2 , a imagem é dividida em partes menores e mais detalhadas, o que permite uma representação mais precisa pelos vetores do codebook. Por isso, as reconstruções apresentam valores de PSNR mais altos, indicando melhor qualidade visual, embora a taxa de compressão seja menor.

Quando blocos de 3×3 são utilizados, ocorre o inverso: como a imagem passa a ter menos blocos, a quantidade total de índices diminui e a taxa de compressão aumenta. No entanto, blocos maiores costumam reunir regiões com mais variação interna, o que prejudica a fidelidade da reconstrução e reduz o PSNR. Assim, confirma-se o trade-off esperado: blocos menores preservam a qualidade, enquanto blocos maiores privilegiam a compressão.

Ao comparar imagens em tons de cinza com imagens coloridas, nota-se que as coloridas apresentam PSNR um pouco menor. Isso acontece porque cada canal (R, G e B) é quantizado separadamente, e as pequenas perdas de cada componente acabam se somando no resultado final. Mesmo assim, as taxas de compressão obtidas foram semelhantes às das imagens em cinza.

Em resumo, os experimentos mostram exatamente o comportamento previsto pela teoria da VQ: blocos menores resultam em maior qualidade reconstruída, enquanto blocos maiores aumentam a compressão, porém com maior perda visual.

3 Segmentação

Nesta etapa do trabalho, foram implementados dois métodos distintos de segmentação aplicados a duas imagens coloridas. O objetivo é comparar abordagens baseadas em diferentes princípios: uma fundamentada exclusivamente em agrupamento no espaço de cores, e outra que utiliza propriedades de textura, incorporando informações adicionais sobre padrões locais da imagem.

A segmentação é uma tarefa fundamental no processamento de imagens, pois permite dividir a imagem em regiões homogêneas que representam objetos, fundos ou áreas de interesse. Neste trabalho, optou-se por métodos que produzem resultados estruturados e que seguem diretamente o que foi solicitado pelo enunciado: um método baseado em agrupamento (k-means) e um método que utilize textura.

Além disso, em ambos os métodos foram testados diferentes valores de K , permitindo avaliar como a granularidade da segmentação influencia a qualidade e o nível de detalhamento obtido.

3.1 Metodologia

A metodologia foi organizada em duas abordagens independentes, cada uma avaliada com três valores distintos de K (2, 3 e 4).

3.1.1 Segmentação por K-means no Espaço de Cor Lab

A imagem é inicialmente convertida do espaço RGB para o espaço Lab, que separa a luminância dos componentes cromáticos e melhora a distinção entre cores semelhantes. Cada pixel é representado pelo vetor (L, a, b) e todos os pixels são reunidos em uma matriz de características.

Em seguida, aplicam-se sucessivamente diferentes valores de K no algoritmo K-means, produzindo segmentações com níveis variados de separação entre regiões. Cada resultado é reorganizado no formato da imagem original, gerando mapas de clusters com diferentes granularidades.

Esse método utiliza apenas a cor como critério de separação, sendo eficaz para segmentar objetos visualmente distintos ou regiões com tonalidades marcadas.

3.1.2 Segmentação baseada em Textura com Filtros de Gabor

Para incorporar informações de textura, utiliza-se um conjunto de filtros de Gabor com múltiplas orientações e frequências espaciais. Essas respostas são capazes de detectar padrões repetitivos e estruturas locais como bordas, rugosidade e variações de orientação.

Após converter a imagem para tons de cinza, cada filtro é aplicado e gera um mapa de textura. Os valores de resposta são vetorizados, normalizados e utilizados como entrada para o algoritmo K-means. Assim como no método anterior, diferentes valores de K são testados (2, 3 e 4), resultando em segmentações mais grossas ou mais detalhadas.

Esse método é capaz de separar regiões que possuem texturas distintas mesmo quando têm cores semelhantes, sendo complementar à segmentação baseada somente em cor.

3.2 Código

3.3 Resultados Obtidos e Análise

Os resultados obtidos mostram diferenças claras entre os métodos de segmentação baseados em cor (K-means no espaço Lab) e aqueles baseados em textura (filtros de Gabor). Além disso, a variação do número de clusters K influencia diretamente o nível de detalhe das regiões segmentadas.

3.3.1 Imagens originais

K-means (Lab) - Imagem da Borboleta

Para a borboleta, o método baseado apenas em cor teve desempenho satisfatório.

K = 2: A segmentação separa de forma nítida o fundo e a borboleta, criando uma divisão simples, porém eficaz. A forma do inseto permanece bem definida.

K = 3: Surge uma subdivisão interna da borboleta, separando áreas mais claras e mais escuras das asas. O fundo também é dividido em mais uma região.

K = 4: A segmentação se torna mais rica em detalhes, incluindo nuances de tons tanto nas asas quanto no fundo. Apesar disso, alguns ruídos começam a aparecer, especialmente na região externa da imagem.

Em geral, o método K-means captou bem as diferenças de cor e luminosidade da borboleta, mesmo sem considerar textura.

3.3.2 Borboleta K-Means

Gabor (Textura) - Imagem da Borboleta

Nos resultados baseados em textura, o comportamento muda significativamente.

K = 2: A segmentação destaca padrões de textura nas asas, mas também introduz ruídos decorrentes das respostas dos filtros.

K = 3: Algumas estruturas internas da borboleta começam a ser separadas com maior consistência, mas o ruído ainda permanece evidente.

K = 4: Surge mais definição na silhueta e nas áreas texturizadas, porém a segmentação ainda mantém regiões ruidosas, especialmente no fundo.

O método baseado em Gabor é muito sensível a variações de textura e, neste caso, acaba segmentando várias regiões que não correspondem a objetos reais, mas sim aos padrões do filtro. Ainda assim, ele ressalta detalhes internos das asas que não aparecem no K-means de cor.

3.3.3 Borboleta Gabor + K-Means

K-means (Lab) - Imagem da Raposa

Para a raposa, que tem variações relevantes de cor entre o animal e o fundo, o método funciona bem.

K = 2: Já é possível separar o animal do fundo, embora algumas partes fiquem agrupadas de forma simplificada.

K = 3: O contorno da raposa aparece de maneira mais fiel, com separação mais clara das sombras e detalhes do pescoço.

K = 4: A segmentação fica mais detalhada, diferenciando ainda mais regiões do pelo e partes do fundo. Porém, assim como na borboleta, o aumento de K também traz pequenas regiões quebradas e ruídos.

O método de cor consegue destacar a raposa de forma eficiente, mostrando uma boa separação entre fundo, pelo claro e pelo escuro.

3.3.4 Raposa K-Means

Gabor (Textura) - Imagem da Raposa

Como na borboleta, o método baseado em textura produz resultados mais irregulares.

K = 2: Os padrões de textura do filtro dominam a imagem, com grande presença de ruído. Ainda assim, algumas áreas da raposa se destacam do fundo. K = 3: A silhueta da raposa torna-se mais perceptível, e diferentes regiões de textura começam a ser agrupadas de maneira mais coerente. K = 4: Surgem regiões mais próximas do formato real do animal, com subdivisão de áreas texturizadas no pelo. No entanto, o ruído permanece significativo.

De forma geral, o método baseado em textura é mais adequado para imagens com padrões repetitivos bem definidos. A imagem da raposa, por ter texturas mais suaves, não se beneficia tanto dos filtros Gabor quanto a segmentação por cor.

3.3.5 Raposa Gabor + K-Means

3.4 Conclusão da Análise

Os experimentos mostram que:

K-means no espaço Lab produz segmentações mais limpas, estruturadas e visualmente coerentes, destacando corretamente os objetos principais (borboleta e raposa).

O método baseado em textura gera segmentações mais fragmentadas, capturando padrões de textura, mas também introduzindo ruídos consideráveis.

A variação de K controla o nível de detalhe: valores baixos criam segmentações mais simples; valores maiores aumentam a granularidade, mas podem introduzir ruído.

Para as imagens utilizadas, a segmentação por cor foi mais eficaz, enquanto o método baseado em textura foi mais útil apenas para ressaltar detalhes internos e padrões finos.

Siglas

DCT Discrete Cosine Transform. 2, 3, 9

IDCT Inverse Discrete Cosine Transform. 3

JPEG Joint Photographic Experts Group. 2, 3, 10

PSNR Peak Signal-to-Noise Ratio. 3, 8–11, 15

VQ Vector Quantization. 11, 15